Linked Lists

COMP 182 · Lecture 05

References, nodes, and list operations

Singly, doubly, and circular linked lists

Array and linked list implementations

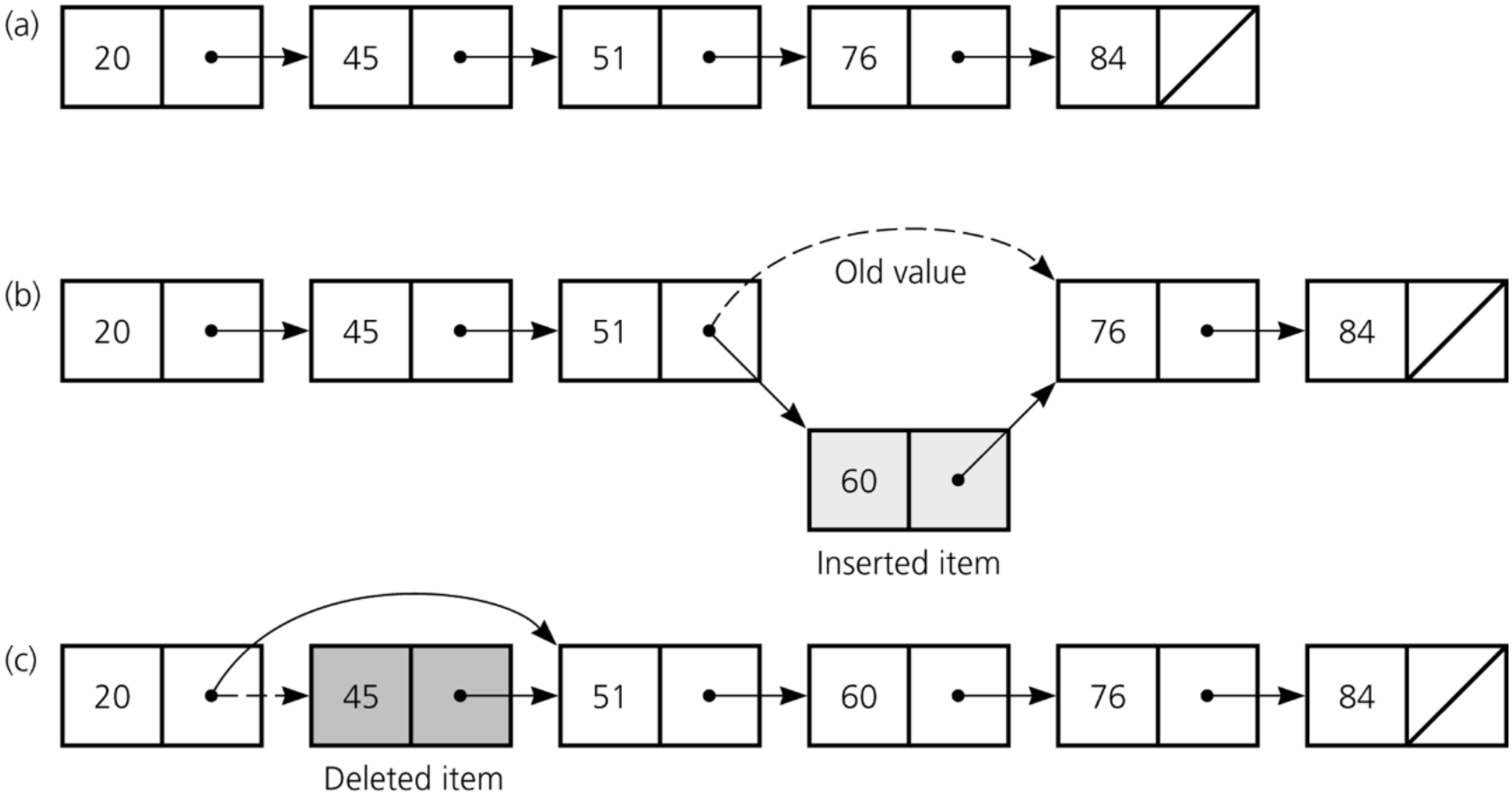
An array stores elements in indexed positions. Its length is fixed.

A resizable array allocates a larger array and copies elements.

A linked list stores nodes connected by references.

Inserting or deleting changes links after the position is found.

Insertion and deletion by relinking



Only neighboring links change. The other elements stay in their nodes.

Java object references

```
Integer intRef = Integer.valueOf(5);
Integer other = null;
```

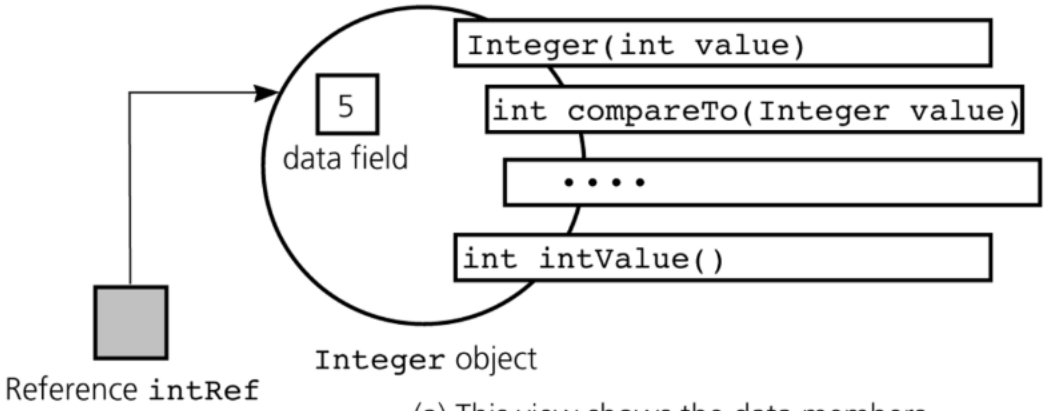
A reference identifies an object or has the value null.

A field of reference type defaults to null.

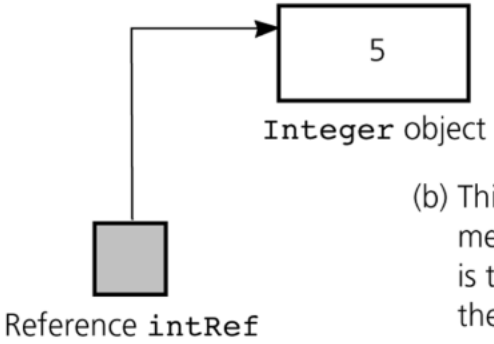
A local variable must be assigned before it is read.

Open Java: ReferenceDemo.java

A reference and its object



(a) This view shows the data members and methods for the object.



(b) This view only shows the data members for simplicity. This is the view used throughout the text.

The reference variable and the referenced Integer object are separate.

Aliasing and garbage collection

```
Integer p = Integer.valueOf(6);
Integer q = p;
p = null;  // q still refers to the object
```

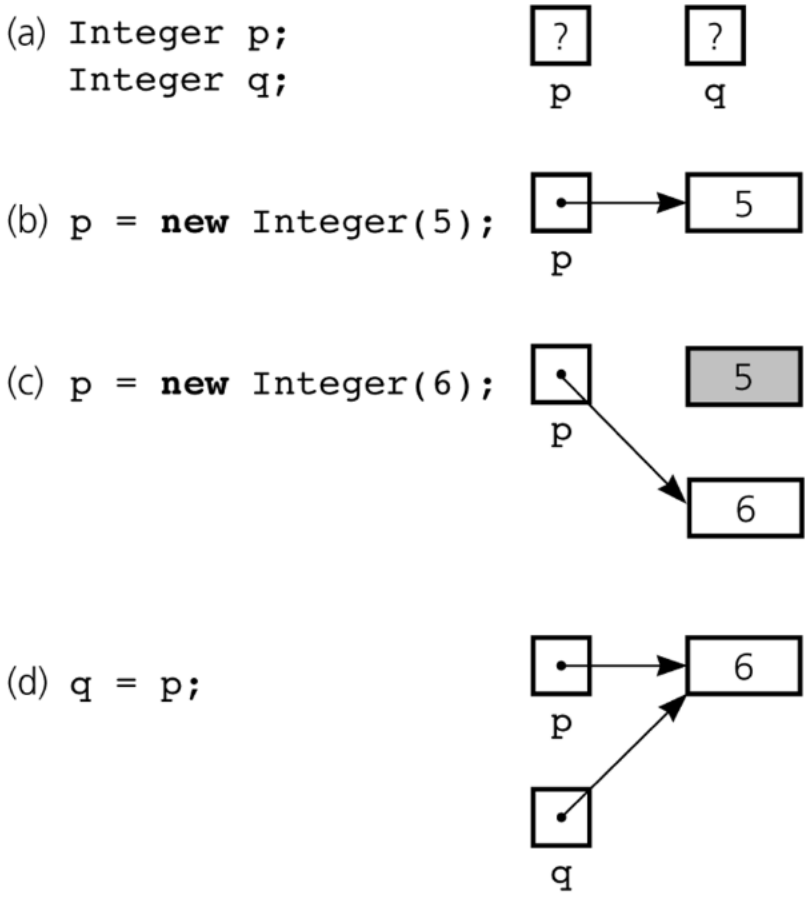
Assigning a reference creates another way to reach the same object.

Changing one reference does not change the other reference.

An unreachable object becomes eligible for garbage collection.

Open Java: ReferenceDemo.java

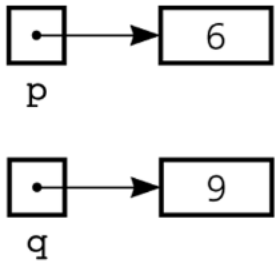
Tracing reference assignments



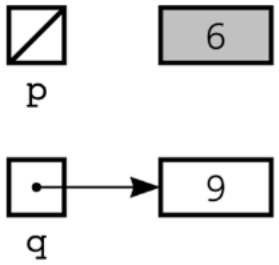
Track each variable separately as objects are allocated and references change.

Reassignment and null

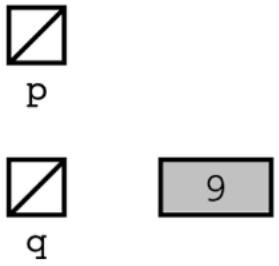
(e) `q = new Integer(9);`



(f) `p = null;`



(g) `q = p;`



Assigning null removes one reference to an object.

Arrays of object references

```
Integer[] scores = new Integer[30];
scores[0] = Integer.valueOf(7);
scores[1] = Integer.valueOf(9);
```

Creating the array allocates reference slots.

Each slot initially contains null.

Assign an object reference before using an object in a slot.

Identity, equality, and method arguments

== compares whether references identify the same object.

equals follows the contract implemented by the class.

Object.equals uses identity unless a class overrides it.

Java passes a copy of each argument value, including references.

Open Java: ReferenceDemo.java

Quiz 1: aliases and mutation

```
Node<String> p = new Node<>("A");
Node<String> q = p;
q.element = "B";
p = null;
System.out.println(q.element);
System.out.println(q.next == null);
```

Predict both lines of output. Explain your reasoning.

Open Java: ReferenceDemo.java

Resizable arrays

The backing array has a fixed capacity.

When capacity runs out, allocate a larger array and copy.

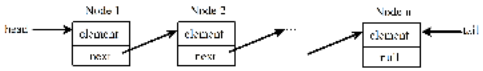
ArrayList appends take amortized O(1) time.

A resize can make one append take O(n) time.

A singly linked node

```
class Node<E> {
  E element;
  Node<E> next;

  Node(E element) {
    this.element = element;
  }
}
```



Each node stores one element and a next reference.

The last node has next == null.

Open Java: Node.java

Building a three-node list

```
Node<String> head = new Node<>("A");
head.next = new Node<>("B");
head.next.next = new Node<>("C");
Node<String> tail = head.next.next;
int size = 3;
```

head refers to the first node. tail refers to the last.

Empty list: head == null, tail == null, size == 0.

Open Java: RecursiveTraversalDemo.java

Traversing a singly linked list

```
Node<E> current = head;
while (current != null) {
  System.out.println(current.element);
  current = current.next;
}
```

Use a separate cursor so head still refers to the first node.

Visiting every node takes O(n) time and O(1) extra space.

Open Java: SinglyLinkedDemo.java

Discussion 1: a safe traversal

A list contains A, null, and C in that order.

What happens if traversal uses head as its cursor?

Should it stop when current.element is null?

Suggest a traversal that preserves the list.

Open Java: SinglyLinkedDemo.java

List state and invariants

```
Node<E> head;
Node<E> tail;
int size;
```

Fields: head, tail, and size.

Empty: head == null and tail == null.

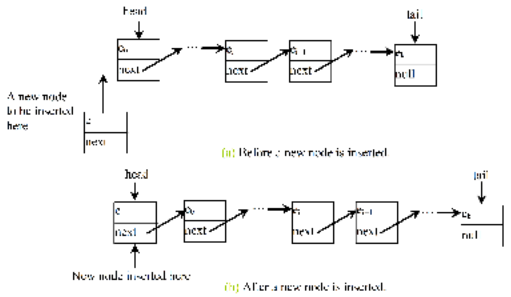
Nonempty: tail.next == null.

The number of reachable nodes equals size.

Open Java: SinglyLinkedList.java

Singly linked addFirst(E e)

```
Node<E> n = new Node<>(e);
n.next = head;
head = n;
if (tail == null) tail = n;
size++;
```



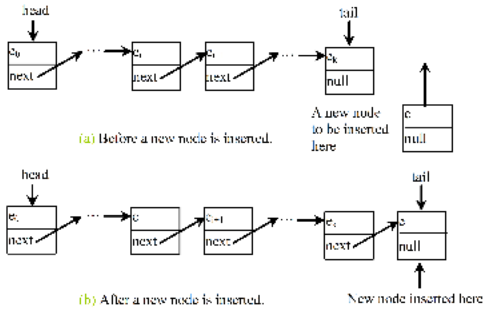
Connect the new node before moving head.

Time: O(1).

Open Java: SinglyLinkedList.java

Singly linked addLast(E e)

```
Node<E> n = new Node<>(e);
if (tail == null) {
  head = n;
} else {
  tail.next = n;
}
tail = n;
size++;
```



A stored tail avoids a traversal.

Time: O(1).

Open Java: SinglyLinkedList.java

Singly linked add(int index, E e)

```
// Require 0 <= index <= size.
if (index == 0) addFirst(e);
else if (index == size) addLast(e);
else {
  Node<E> prev = nodeAt(index - 1);
  Node<E> n = new Node<>(e);
  n.next = prev.next;
  prev.next = n;
  size++;
}
```

Locate the predecessor, then change two links.

Interior insertion by index: O(n).

Open Java: SinglyLinkedList.java

Quiz 2: insertion order

Start with A, B, C. prev refers to A.

```
Node<String> n = new Node<>("X");

prev.next = n;

n.next = prev.next;
```

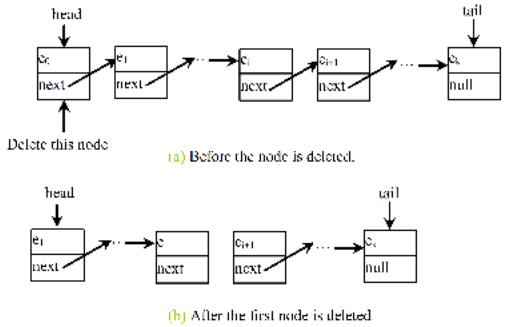
What is reachable from A after these statements?

Rewrite the two assignments to insert X correctly.

Open Java: SinglyLinkedList.java

Singly linked removeFirst()

```
if (head == null)
  throw new NoSuchElementException();
E value = head.element;
head = head.next;
if (head == null) tail = null;
size--;
return value;
```



Removing the only node must clear tail too.

Time: O(1).

Open Java: SinglyLinkedList.java

Singly linked removeLast()

```
if (head == null)
  throw new NoSuchElementException();
if (head == tail) return removeFirst();
Node<E> prev = head;
while (prev.next != tail)
  prev = prev.next;
E value = tail.element;
prev.next = null;
tail = prev;
size--;
return value;
```

The predecessor of tail requires a traversal.

Time: O(n).

Open Java: SinglyLinkedList.java

Singly linked remove(int index)

```
// Require 0 <= index < size.
if (index == 0) return removeFirst();
Node<E> prev = nodeAt(index - 1);
Node<E> victim = prev.next;
prev.next = victim.next;
if (victim == tail) tail = prev;
size--;
return victim.element;
```

Find the predecessor and bypass the target.

Time: O(n).

Open Java: SinglyLinkedList.java

Discussion 2: boundary cases

An empty list receives addLast("A").

Then removeFirst() removes its only node.

What must head, tail, and size become?

What should a second removeFirst() do?

Open Java: LinkedListChecks.java

Array and linked list tradeoffs

Operation	Resizable array	Singly linked list
Indexed access	O(1)	O(n)
Add at front	O(n)	O(1)
Append with tail	Amortized O(1)	O(1)
Remove at end	O(1)	O(n)
Extra storage	Unused capacity	One next link per node

Open Java: SinglyLinkedList.java

Finding a position versus changing links

Searching by value or index usually takes O(n).

Changing links at a known position can take O(1).

A singly linked deletion needs the predecessor.

No shifting does not guarantee a faster operation.

Open Java: SinglyLinkedList.java

Passing a list reference to a method

```
void advance(Node<E> node) {
  node = node.next;  // local reassignment
}

void skipNext(Node<E> node) {
  node.next = node.next.next;
}
```

A copied reference can still reach and mutate shared nodes.

Reassigning a parameter does not reassign the caller's head.

Open Java: ReferenceDemo.java

Quiz 3: copied references

Start with head referring to A, B, C.

```
advance(head);   // node = node.next

skipNext(head);  // node.next = node.next.next
```

What does head.element contain?

Which nodes remain reachable from head?

Why do these methods affect the caller differently?

Open Java: ReferenceDemo.java

Recursive traversal

```
void forward(Node<E> n) {
  if (n == null) return;
  System.out.println(n.element);
  forward(n.next);
}

void backward(Node<E> n) {
  if (n == null) return;
  backward(n.next);
  System.out.println(n.element);
}
```

Print before recursion for forward order.

Print after recursion for reverse order.

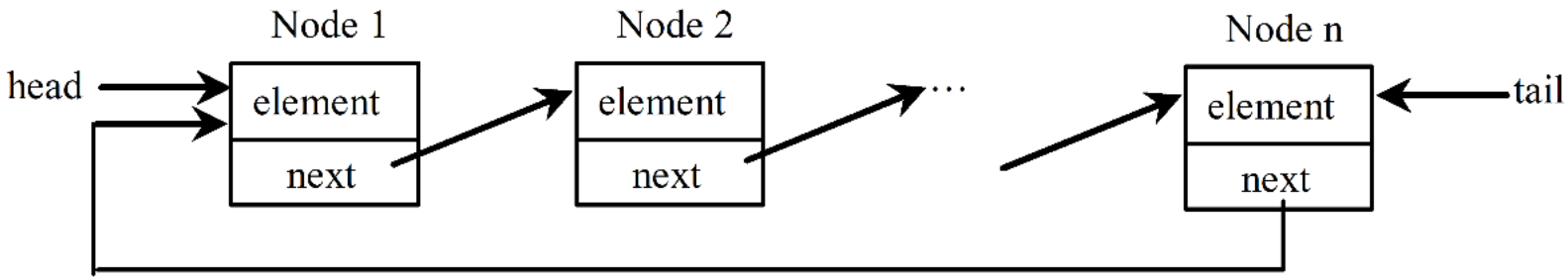
Open Java: RecursiveTraversalDemo.java

Circular singly linked lists

For a nonempty list, tail.next == head.

Traversal stops when the cursor returns to its starting node.

An empty list still needs a separate check.



Open Java: CircularListsDemo.java

Quiz 4: circular traversal

A nonempty circular list links tail.next to head.

Why does while (current != null) fail to finish?

Write a loop that visits each node once.

How should it handle an empty list and one node?

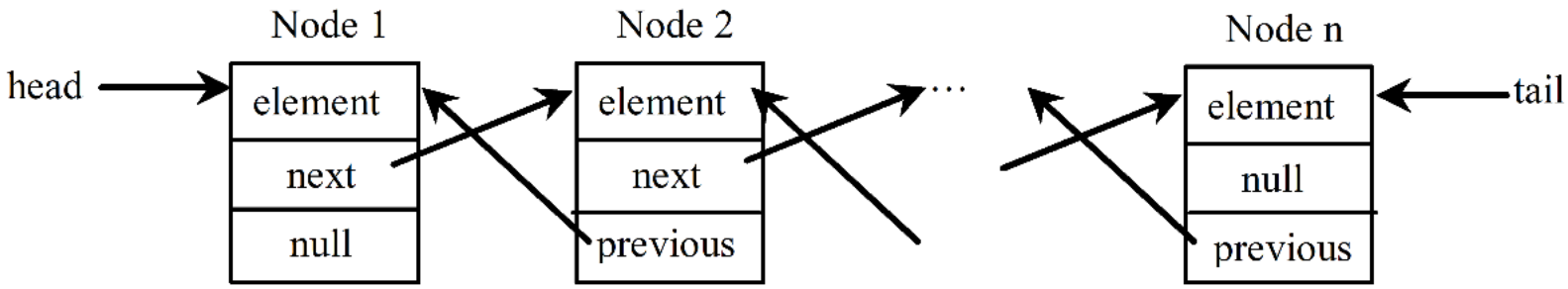
Open Java: CircularListsDemo.java

Doubly linked lists

Each node stores element, next, and previous.

Traverse forward from head or backward from tail.

For a linear list: head.previous == null and tail.next == null.



Open Java: DoublyLinkedList.java

Doubly linked addFirst(E e)

```
Node<E> n = new Node<>(e);
n.next = head;
if (head == null) {
  tail = n;
} else {
  head.previous = n;
}
head = n;
size++;
```

The new node becomes head.

The old head links back to it.

Time: O(1).

Open Java: DoublyLinkedList.java

Doubly linked removeFirst()

```
if (head == null)
  throw new NoSuchElementException();
E value = head.element;
head = head.next;
if (head == null) tail = null;
else head.previous = null;
size--;
return value;
```

Clear the new head's backward link.

One node becomes an empty list.

Time: O(1).

Open Java: DoublyLinkedList.java

Doubly linked addLast(E e)

```
Node<E> n = new Node<>(e);
n.previous = tail;
if (tail == null) {
  head = n;
} else {
  tail.next = n;
}
tail = n;
size++;
```

The new node becomes tail.

The old tail links forward to it.

Time: O(1).

Open Java: DoublyLinkedList.java

Doubly linked removeLast()

```
if (tail == null)
  throw new NoSuchElementException();
E value = tail.element;
tail = tail.previous;
if (tail == null) head = null;
else tail.next = null;
size--;
return value;
```

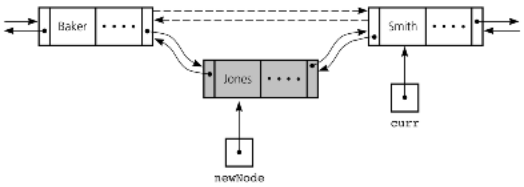
The previous link gives direct access to the new tail.

Time: O(1).

Open Java: DoublyLinkedList.java

Insert before a known node

```
// curr is a node in this list.
Node<E> n = new Node<>(e);
n.previous = curr.previous;
n.next = curr;
if (curr.previous == null) head = n;
else curr.previous.next = n;
curr.previous = n;
size++;
```

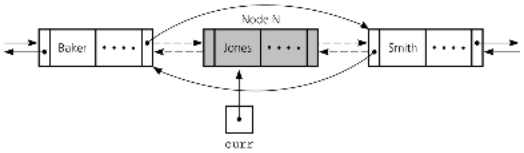


Known curr: O(1).
Finding curr by index: O(n).

Open Java: DoublyLinkedList.java

Remove a known node

```
// curr is a node in this list.
Node<E> p = curr.previous;
Node<E> q = curr.next;
if (p == null) head = q;
else p.next = q;
if (q == null) tail = p;
else q.previous = p;
E value = curr.element;
curr.next = curr.previous = null;
size--;
return value;
```



Open Java: DoublyLinkedList.java

Discussion 3: both directions

A doubly linked list contains A, B, C.

Remove the known node B.

Which two neighboring references must change?

How do removal of A, C, and the only node differ?

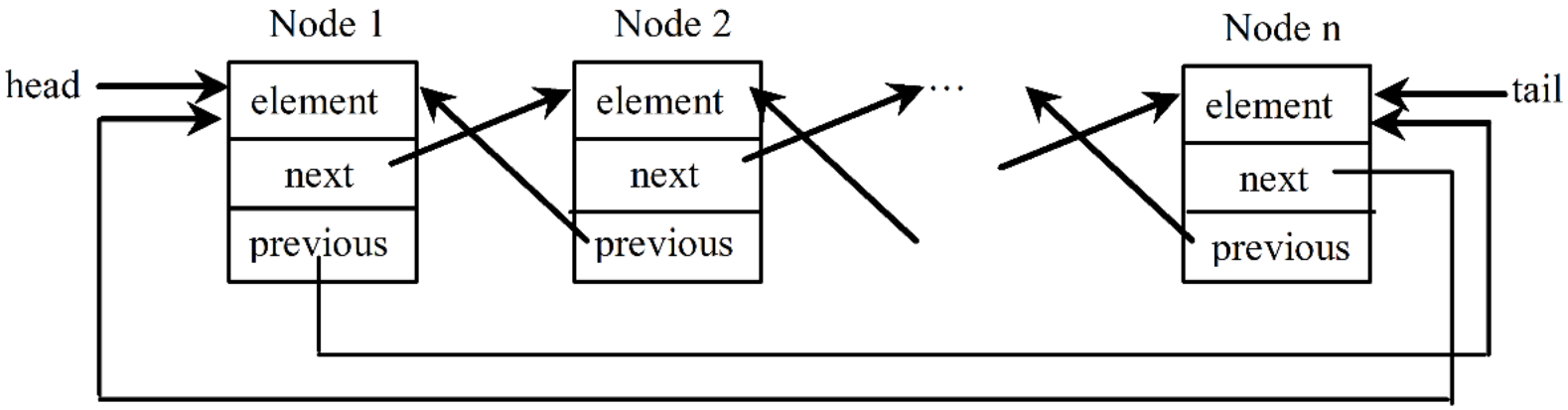
Open Java: DoublyLinkedDemo.java

Circular doubly linked lists

For a nonempty list, tail.next == head.

The backward link also wraps: head.previous == tail.

For one node, both links point back to that node.



Open Java: CircularListsDemo.java

Java Collections Framework: LinkedList

java.util.LinkedList<E> implements List<E> and Deque<E>.

Its implementation uses doubly linked nodes.

End operations support queue and deque use.

Indexed operations still require traversal.

Open Java: JavaLinkedListDemo.java

Using java.util.LinkedList

```
LinkedList<String> names = new LinkedList<>();
names.addLast("Baker");
names.addFirst("Adams");
String first = names.getFirst();
String removed = names.removeLast();

for (String name : names) {
  System.out.println(name);
}
```

Import java.util.LinkedList.

Use an iterator when processing the list sequentially.

Open Java: JavaLinkedListDemo.java

ArrayList and LinkedList operation costs

Operation	ArrayList	LinkedList
get(i) / set(i, e)	O(1)	O(n)
Append	Amortized O(1)	O(1)
Add / remove at front	O(n)	O(1)
Remove at end	O(1)	O(1)
Find by value	O(n)	O(n)
Insert / remove by index	O(n)	O(n)

LinkedList iterator insertion or removal is O(1) after positioning.

Open Java: JavaLinkedListDemo.java

Exit quiz: choosing list operations

1. Cost of singly linked removeLast(), even with tail?
2. Cost of unlinking a known doubly linked node?
3. Cost of finding that node by index?
4. Why can repeated get(i) make traversal $O(n^2)$?
5. Which traversal would you use instead?

Open Java: JavaLinkedListDemo.java

Java examples

Node.java

ReferenceDemo.java

SinglyLinkedList.java

SinglyLinkedDemo.java

DoublyLinkedList.java

DoublyLinkedDemo.java

RecursiveTraversalDemo.java

CircularListsDemo.java

JavaLinkedListDemo.java

LinkedListChecks.java